

Contents

1	Introduction	1
2	Monitoring	1
2.1	Design and architecture	1
2.2	What we monitor	1
2.3	Reflections	1
3	Logging	2
3.1	Design and architecture	2
3.2	What we log	2
3.3	Reflections	2

1 Introduction

ITU-MiniTwit is a Twitter-like microblogging service from the original Flask codebase that this course provides. Over the semester, we refactored it into a Go application backed by PostgreSQL, deployed in containers on a single-node Docker Swarm hosted at Hetzner. We had three main motivations for picking go: We wanted to challenge ourselves by doing something new, prioritized the performance of a pre-compiled language, and the static typing of go seemed accesible. The rest of this report describes how we provisioned, built, deployed, observed, and secured the system.

2 Monitoring

2.1 Design and architecture

Our metrics pipeline has three components: the Go web service produces metrics (`GET /metrics`), Prometheus pulls and stores them, and Grafana visualizes them. A fourth service, node-exporter, exposes host-level metrics (CPU, memory, disk). We used the official `prometheus/client_golang` library. Our HTTP middleware automatically records request count, latency, and status code for every route. The counters for specific stats are incremented separately inside the relevant handlers and exposed as, for instance, `new_registered_users`. Prometheus then scrapes the metrics every 15s, and Grafana pulls it and visualizes it. The Grafana dashboard is fully provisioned from a folder in our repository, which defines what the dashboard looks like. It is set up automatically after deployment, with the tradeoff that any changes from the UI will not be persistent. Lastly, Caddy exposes Grafana at `grafana.kulturbase.dk` over HTTPS.

2.2 What we monitor

The main metrics we monitor are request rate, error rate, latency, and success rate. The reason behind this is that it immediately tells us whether something is wrong, and if so, points us in that direction. The second part of the dashboard focuses on overall health and serving capabilities. The main visualizations show the CPU load, health check, and an overview of activity on the server. This enables us to distinguish app issues from infrastructure issues and whether the app is being actively used.

2.3 Reflections

The monitoring setup needs to take into consideration the deployment strategy. When switching from `docker compose` to `Docker Swarm` with multiple replicas, our scraping silently broke. Each scrape hit a different replica through Swarm's load-balancing virtual IP, so counters appeared to reset between scrapes. The solution was to use the Prometheus DNS service discovery to scrape each replica separately, and then sum the results in Grafana.

3 Logging

3.1 Design and architecture

Our logging pipeline uses an EFK stack (Elasticsearch, Filebeat, Kibana), as presented in the exercises. The Go web service writes structured JSON logs through `slog` to stdout. Docker captures container stdout into `/var/lib/docker/containers/` via its default JSON-file driver. Filebeat runs one instance per node in swarm, tails those log files, decodes the Docker JSON, and ships entries directly to Elasticsearch, which stores and indexes them. Kibana sits in front for search and visualization. The whole stack runs on its own elk overlay network, with Caddy exposing `kibana.kulturbase.dk` over HTTPS.

3.2 What we log

The Go app produces structured JSON logs at multiple levels through `slog`. The HTTP middleware automatically logs every request with method, route, status, duration, and client IP. Handler-level `Logger.Error` calls add explicit context when something fails (DB query, template render, JSON decode). Container stdout from supporting services (Postgres, Caddy, Prometheus) is also captured automatically. In Kibana, we built a dashboard with four saved searches: live error stream, failed-request stream (`status >= 400`), user-activity stream (registrations, logins, posts), and overall log volume by service.

3.3 Reflections

Choosing EFK stack over Grafana + Loki was, in our opinion, the wrong call for a project this small. Elasticsearch alone consumes a large amount of memory, and the stack was tricky to set up. For example, Filebeat refused to start until its config file was owned by root with the right permissions, which broke our deploys until we fixed the file ownership manually on the server. Loki would have let us keep everything under one Grafana frontend, removing Kibana as a separate system to configure and share access to. Even though it introduced unnecessary complexity, the decision was still worth it as a learning experience, as we got to try the industry-standard logging stack and ran into the kinds of real-world problems. For a project of this size it was overkill, but as a one-time exercise to engage with the heavier tools, it was a good choice. It also nicely separated the two concerns - Grafana telling us whether something is wrong, and Kibana explaining what happened.